(e.g., Go, Java, Node.js, and Python) and a range of profile types, including CPU time, heap, and threads.

Then, there are several ISVs that either bundle profiling or have standalone offerings. Some have been released in a source-available manner, including but not limited to the following:

- The Datadog profiler (https://docs.datadoghq.com/profiler/), part of the overall offering, enables you to find CPU, memory, and I/O bottlenecks; the profiles are broken down by function, class name, and line number in your code.
- Dynatrace's offering (http://mng.bz/5w5O), part of their overall suite, allows you to do CPU profiling, including drilling down to the function level, memory dump analysis, and process crashes.
- g:Profiler (https://granulate.io/continuous-profiling/) is an SaaS offering (but also available as open source) supporting AWS, Azure, and Google Cloud environments. It supports an array of languages (Clojure, Scala, C/C++, Ruby, Java, Python, JavaScript, PHP, and Go), uses eBPF, and claims a less-than-1% utilization penalty.
- New Relic has a feature called Real-time profiling for Java (http://mng.bz/OP6n) that supports flame graphs as well as the Thread profiler tool https://docs.newrelic.com/docs/apm/apm-ui-pages/events/thread-profiler-tool/ that you can use for Java, .NET, Python, and Ruby.
- Prodfiler (https://prodfiler.com/) touts itself as the distributed lightweight continuous whole-system profiler, enabling you to get insights into CPU usage, supporting flame, requiring Linux Kernel with version 4.15 or above, and available in platforms such as Kubernetes and AWS ECS.
- Splunk offers the AlwaysOn (http://mng.bz/5w5O) profiler.

A great overview and lookup for the continuous profiling space is Mark Hansen's Profilerpedia (https://profilerpedia.markhansen.co.nz/), a catalog of tools and profiler data formats. If you want to dive deeper into the topic, consider checking out *The Busy Developer's Guide to Go Profiling, Tracing and Observability* (https://github.com/DataDog/go-profiler-notes/tree/main/guide) from Datadog.

9.1.5 Using continuous profiling to assess continuous profiling

In this subsection, we will cover using continuous profiling to measure resource usage of an OpenTelemetry collector. I'm using the OpenTelemetry collector here as an example CP source, since it readily exposes `pprof`-formatted profiles, but you could really plug in any other HTTP API offering profiles (e.g., our Go example from the beginning of the chapter).

To generate profiles, we will set up a signal source (as usual, we will use `holly` with a load generator) that sends traces to an OpenTelemetry collector, and then we will use Parca to receive and query profiles from the collector. All the sources are in